

AI 求职 MVP 项目计划(v2:Next.js版)

定位:全栈 + 本地AI部署 + Agent安全,面向澳洲市场求职作品集 背景:2014毕业,2016起接FE,2021起full stack(PHP/SQL/HTML/CSS/JS/jQuery/API/webhook);私人项目用过React;另会Laravel、CodeIgniter、Vite、Firebase、MongoDB、AWS;偏好TypeScript

一、项目定位与叙事

核心故事线:

“5年PHP/SQL full stack经验 + 主动转型AI应用开发。自建本地优先(on-prem)的AI技术栈,涵盖RAG、agent、vision LLM,并针对agent权限边界和隔离安全做了专门设计——对应企业对数据主权和agent失控风险的真实关切。”

目标岗位方向(同时投,不押单一方向):

1. Full-Stack Developer(标注AI项目为差异化)
2. AI/LLM Application Engineer(中级,非junior)
3. Agent安全 / On-prem AI部署方向(细分差异化亮点)

薪资参考区间(澳洲,仅供参考,以官方/招聘市场实时信息为准):

- Full-Stack + AI加成:AUD 130K–160K
 - AI/LLM应用方向中级岗:AUD 140K–170K
-

二、技术栈总览

已有基础

- Ollama(本地LLM推理)
- ComfyUI + Stable Diffusion(文生图)
- Docker(容器化)
- FastAPI(后端)
- openclaw(agent功能)

前端(v2调整: Astro → Next.js)

调整原因:项目动态模块占比高(user管理/CTE/chatbot/RAG查询都需要交互),Astro的静态优势用不上;且你已有React实战经验,Next.js学习成本更低,内置routing/navigation/TS,省掉裸React当年要自装一堆库的麻烦。

- **框架:**Next.js(App Router)+ TypeScript(`create-next-app` 一键配置,不需要额外装路由/导航库)
- **样式:**Tailwind CSS
- **UI组件**(如需现成组件,配Tailwind不冲突):shadcn/ui(不用MUI,避免和Tailwind样式哲学打架)
- **CTE(点击定位编辑):**GrapesJS(内置resize/dnd/style manager,不手写编辑器引擎)
- **表单:**react-hook-form + Zod(不用Yup,Zod对TS更友好且和react-hook-form是标配组合)
- **全局状态:**优先React Context,规模变大再上Zustand(不用Redux,MVP规模属过度设计)
- **动画/交互:**GSAP + ScrollTrigger(滚动动画)、Swiper.js(图片轮播)
- **图标:**lucide-react
- **图表:**Chart.js 或 Recharts(优先于D3,除非有高度定制需求)
- **测试**(如需要):Vitest(比Jest更轻量快速;MVP阶段可先跳过,面试时能讲清楚取舍即可)

后端与数据

- **数据库:**Postgres + pgvector,或 MongoDB Atlas Vector Search(你已熟悉MongoDB,两者均可,依你偏好选一个,不必强求Postgres)
 - **ORM/Migration**(若选Postgres路线):SQLAlchemy + Alembic(协作与schema演进必备)
 - **Auth:**JWT + RBAC(owner/admin/user)。你已熟悉Firebase → 优先考虑 **Firebase Auth**(自带邮箱/OAuth登录+JWT+custom claims做角色),比重新学fastapi-users/Supabase更省力
 - **Embedding:**本地轻量模型(如 nomic-embed-text,经Ollama跑),与对话生成LLM分离
 - **Vision LLM:**用于图生CSS(预期需人工微调,不追求全自动)
 - **Chatbot:**替代传统contact form,承担留资/转人工功能,建议做流式响应(SSE/WebSocket)
 - **部署:**AWS(你已有经验,可用于托管本地跑不动的服务、S3存文档等)
 - **在线AI接口预留:**本地模型 ↔ 云端模型切换接口(合规/成本权衡)
-

三、核心功能模块

1. **Landing Page / About Us / User 页面**(Next.js,内容页用SSG静态生成保持加载速度/SEO)
 2. **CTE 点击定位编辑**:点击元素弹出编辑框(定位用 `getBoundingClientRect()`),支持改文字、图片、padding/margin(经GrapesJS Style Manager)
 3. **RAG 企业文档问答**:
 - 摄取pipeline:解压zip → 解析(pdf/docx/md等)→ 清洗 → chunking → embedding(本地小模型)→ 存入向量库
 - 查询API:检索 → 组装prompt → LLM生成 → 返回时附带来源引用(RAG核心卖点,面试必考点)
 4. **Chatbot**:对话式交互,LLM返回带 `type` 字段的结构化JSON(text/radio/checkbox/select),前端动态渲染对应表单控件;内置意图识别(关键词/规则即可),触发时提取结构化信息(邮箱/需求)存入CRM占位接口,支持转人工
 5. **RBAC 权限系统**:JWT携带role字段;前端按role控制UI显示;后端每个接口独立做角色校验,不能仅靠前端隐藏
 6. **Agent Skill(openclaw)安全边界**:
 - 明确列出agent可调用的工具、可访问的文件系统范围
 - 关键操作加人工确认机制
 - 记录agent行为日志,便于审计
 7. **Docker隔离说明**:服务间网络/文件系统访问范围文档化
 8. **第三方API占位**:CRM接口、在线AI fallback接口(先留桩,demo时说明设计,不需要完全打通)
-

四、开发顺序建议(避免任务过散,按模块拆session)

阶段1:骨架

- Next.js(App Router + TS)+ Tailwind 基础页面结构
- 数据库初始化(Postgres+SQLAlchemy+Alembic,或MongoDB),users表/集合(id/email/role等)
- Firebase Auth 接入 + JWT + RBAC后端校验

阶段2:RAG核心

- 摄取pipeline脚本(zip→解析→chunk→embedding→存入向量库)
- 查询API(含来源引用)
- 本地embedding模型接入(nomic-embed-text)

阶段3:Chatbot

- 对话界面(React组件,流式响应)
- 结构化输出驱动UI(radio/checkbox/select动态渲染)
- 意图识别 + 结构化信息提取 + 转人工/CRM占位

阶段4:CTE编辑器

- 集成GrapesJS
- 支持文字/图片/padding/margin编辑

阶段5:视觉增强(后置,非核心)

- GSAP滚动动画、Swiper轮播
- Vision LLM图生CSS(预期人工微调)

阶段6:Agent安全层

- openclaw权限边界文档化
- 行为日志 + 简单红队测试(提示注入、越权访问测试)

五、Claude Code 使用策略

- 按模块拆session,不要一次性把所有需求丢给一个对话
- IDE:VS Code + Claude Code官方扩展(不必须用Cursor)
- 订阅:先用Pro(\$20/月)跑1-2个模块, /usage 观察消耗速度;若频繁撞5小时/周限额,再评估升级Max 5x(\$100/月)
- 明确给指令时指定技术选型(如"用Next.js App Router + shadcn/ui做这个页面"),避免它自己造轮子或引入冲突的库(如MUI)

六、面试准备要点(自查清单)

- 能讲清楚RAG的检索质量怎么评估、chunk策略怎么定
- 能讲清楚RBAC的后端校验逻辑,以及“绕过前端直接调API”场景怎么防
- 能讲清楚为什么选RAG不选fine-tuning(数据更新频率、可追溯性)
- 能讲清楚agent权限边界设计,对应OpenAI/HuggingFace沙箱逃逸事件的思考
- 能讲清楚系统扩展瓶颈(文档量从100MB到10GB,哪里先扛不住)
- 能讲清楚为什么从Astro改用Next.js(动态内容占比、已有React经验、开发效率)
- 能讲清楚用AI辅助开发(Claude Code)的边界:架构设计和技术选型是你主导的